# Statement of Purpose

*Daneshvar Amrollahi* *Computer Science Ph.D.*

## Introduction

Several examples of groundbreaking work in computer science are based on mathematically rigorous approaches. A remarkable example is the revolutionary Turing Award-winning RSA algorithm that is based on the factorization of large primes, an appealing problem to several 19th-century mathematicians when a computer did not even exist. I am intrinsically motivated by the appearance of mathematical concepts in areas of computer science that might seem unconnected at first sight. A personal example is the appearance of ideals from ring theory in my work on loop invariants. I believe combining existing mathematical theories (SMT solving, first-order logic, automata theory) with computer science concepts could lead to extraordinary results, and this is a skill that I wish to develop during my Ph.D. studies at Stanford.

## Research Experience

### TU Wien Internship

My first research experience happened during a 7-month internship at the Automated Program Reasoning group at TU Wien under the supervision of Prof. Laura Kovács and Prof. Ezio Bartocci. I contributed to two research projects leading to valuable results and publications.

The first project focused on polynomial invariant generation. This topic is well understood for a restricted class of 'solvable' loops, but very little is known outside this restrictive computational model. In fact, in previous works, the approach is limited to only linear combinations over loop variables. However, I advanced the state-of-the-art methodology for loop invariant generation by establishing an approach to automatically synthesize invariants for unsolvable loops by considering not only linear combinations over loop variables but also polynomial relations among them. Our approach also applies to probabilistic loops. The work led to a conference publication at the *29th Static Analysis Symposium (SAS'22)* and won the *Radhia Cousot Young Researcher Best Paper* award. My contributions include designing a polynomial-time graph algorithm to characterize the program variables responsible for unsolvability, introducing a new technique for synthesizing invariants in unsolvable loops, and fully implementing our theories in Python to have them automated. During the publication process, I recognized the importance of communicating key concepts and ideas to a broader audience of computer scientists.

In the second project, I worked on algebraic reasoning for loop synthesis. This research established a sound and complete SMT-based approach for synthesizing loops satisfying a set of polynomial loop invariants, combining symbolic computation techniques over algebraic recurrence equations with polynomial constraint solving. In this project, I worked independently, and I was in charge of building up the experiments. Additionally, I discovered a potential application of the work in compiler optimization, which was synthesizing equivalent single-path programs with less expensive operations. The paper got published in the ACM journal *Formal Aspects of Computing*. This second project was also successful: I further developed my technical writing skills and demonstrated my ability to conduct independent research with minimal oversight.

### ETH Zürich Internship

After my internship at TU Wien, I started another internship at ETH Zurich to work on more research problems and get to know the field of formal methods better. At ETH, I started working on verifying initialization code in Golang. Initialization code is used to ensure that a package or library is in some desired state at the beginning of the program execution. In Golang, if we want to verify initialization code regardless of the order of the files passed to the compiler, we face a concurrency problem. I came up with a non-trivial solution (not considering all possible file orderings) to this problem by employing the Viper intermediate verification language and separation logic. The idea was based on finding a preserving invariant for each initialization block. I also worked on translating our problem to a programming model introduced by Peter W. O'Hearn by statically analyzing the source code of the Go program. However, designing a scalable approach applicable to real-world programs with thousands of lines of code and hundreds of files is difficult due to nested package dependencies and the high number of possible interleavings, and has not been done so far to the best of my knowledge. I would like to develop a strong generalized framework for dealing with concurrency problems in the area of distributed systems in a scalable manner.

### EPFL Internship

Having experienced the theoretical side of formal methods, I decided to do an internship at the Dependable Systems Lab (DSLAB) at EPFL under the supervision of Prof. George Candea to see how these methods can be applied to real-world computer systems. Ensuring the reliability and correction of computer systems becomes challenging when it comes to real-world applications. Automated verification can be used as a powerful tool for this matter. At DSLAB, they had already done a lot of work in verification of network functions (NFs) using symbolic execution. However, a major problem with symbolic execution is that it runs into the path explosion problem, especially when dealing with larger (real-world) programs with loops being a major source. I mitigated this by summarizing loops in 9 widely used string functions in the standard C library using quantifiers in first-order logic. I did this by integrating Z3's support for quantifiers into KLEE's symbolic execution engine source code. As a result, this decreased the number of paths from $\mathcal{O}(n)$ to $\mathcal{O}(1)$ in each case. The outcome of my work not only helps with the scalability of verifying NFs but also mitigates the path explosion problem when doing symbolic execution on almost any C code using string operations from the standard C library. Even though this was only a prototype, the implications of this work were vast, and for the first time, I felt the joy of the applicability of my work to software with a high impact.

## Why Stanford?

Overall, my research interests align very well with that of Prof. Barrett, Prof. Aiken, and Prof. Trippel. I found the work of Prof. Barrett on SMT Solving and his trend toward neural network verification a good fit. Prof. Aiken's recent work on invariant generation (POPL'22 and TACAS'22) perfectly matches my research experience and interests. I am also interested in working with Prof. Trippel on leveraging techniques from formal methods to verify real-world hardware systems (e.g., data centers). The level of expertise I will earn after finishing the world-leading Computer Science Ph.D. program at Stanford will excellently prepare me for a research-based career at a top industrial company, research institute, or university, which I strongly desire.